Simulation de la propagation d'un faisceau gaussien et introduction à l'effet Kerr optique

NDIAYE Babacar

1 Introduction

Les faisceaux laser jouent un rôle central en optique moderne, notamment dans les domaines de la photonique, des télécommunications et de la caractérisation des matériaux. Parmi les modèles les plus utilisés, le faisceau gaussien constitue une solution fondamentale de l'équation de propagation dans l'approximation paraxiale.

Dans les milieux non linéaires, les propriétés du faisceau peuvent être modifiées par des effets dépendant de l'intensité lumineuse, comme l'effet Kerr optique. Ce phénomène conduit à une variation de l'indice de réfraction du milieu, influençant la propagation du faisceau.

L'objectif de ce projet est de simuler numériquement la propagation d'un faisceau gaussien et d'étudier qualitativement l'influence de l'effet Kerr sur son comportement.

2 Objectifs

Les objectifs de cette étude sont :
- Modéliser un faisceau gaussien en propagation
- Étudier l'évolution du rayon du faisceau avec la distance
- Visualiser la distribution d'intensité
- Introduire un modèle simplifié de l'effet Kerr
- Analyser l'impact de la non-linéarité sur la propagation

3 Modèle théorique

3.1 Faisceau gaussien

$$I(r,z) = I_0 \frac{w_0^2}{w(z)^2} \exp\left(-\frac{2r^2}{w(z)^2}\right)$$

avec :
- w_0 : rayon au waist
- $w(z)$: rayon du faisceau à la position z

3.2 Évolution du rayon

$$w(z) = w_0 \sqrt{1 + \left(\frac{z}{z_R}\right)^2}$$

$$z_R = \frac{\pi w_0^2}{\lambda}$$

est la longueur de Rayleigh.

3.3 Effet Kerr optique

$$n = n_0 + n_2 I$$

avec :
— n_2 : coefficient non linéaire
— I : intensité
Cela entraîne une modification de la propagation (auto-focalisation).

4 Méthodologie

4.1 Simulation numérique

Le faisceau est simulé sur une grille spatiale en coordonnées radiales et longitudinales.

4.2 Paramètres

— Longueur d'onde : $\lambda = 633\,nm$
— Waist initial : $w_0 = 1\,mm$
— Domaine de propagation : $z \in [-0.1, 0.1]\,m$

4.3 Cas étudiés

— propagation linéaire
— propagation avec effet Kerr simplifié

5 Résultats et analyse

5.1 Profil du faisceau

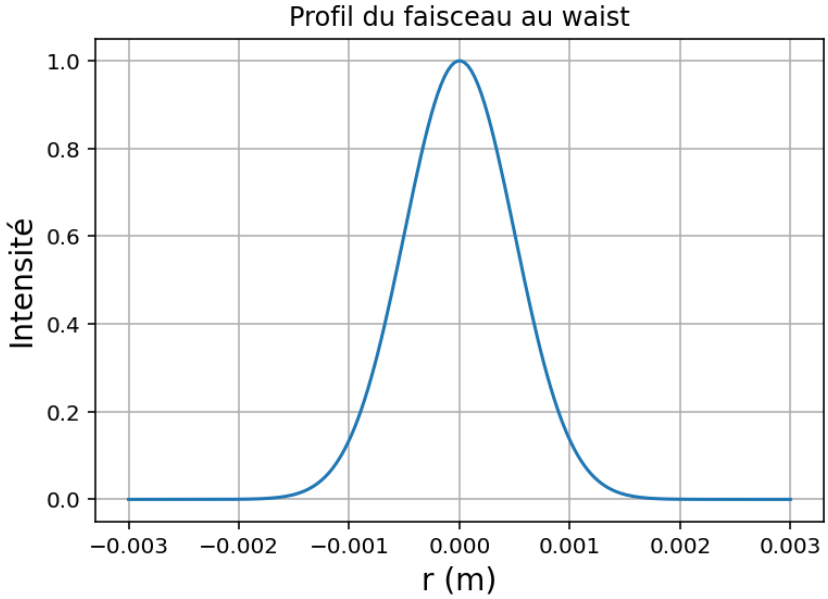

FIGURE 1 – Distribution d'intensité du faisceau gaussien

Le faisceau présente une distribution symétrique caractéristique, avec un maximum au centre.

5.2 Évolution du rayon

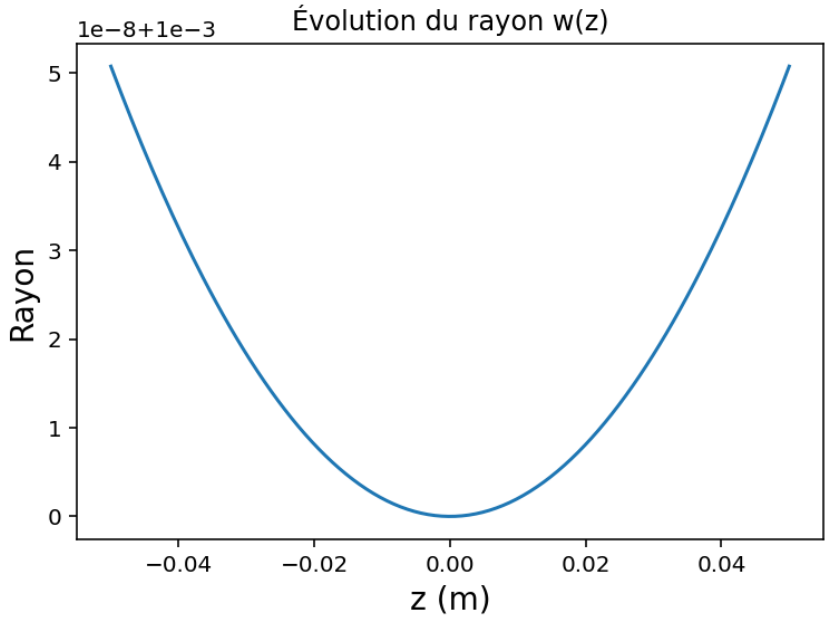

FIGURE 2 – Évolution du rayon $w(z)$

Le faisceau diverge progressivement après le waist, conformément à la théorie.

5.3 Propagation 2D

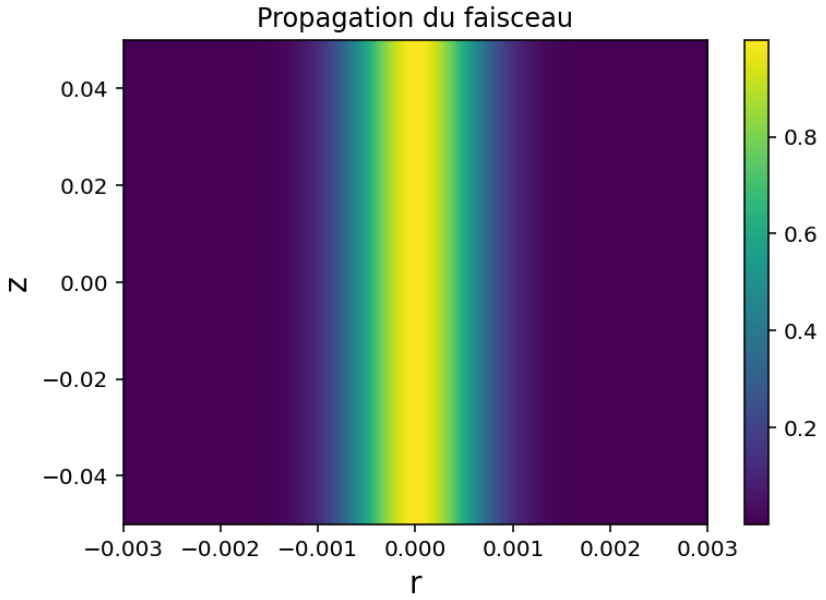

FIGURE 3 – Propagation du faisceau en fonction de z

On observe la divergence du faisceau loin du point focal.

5.4 Effet Kerr

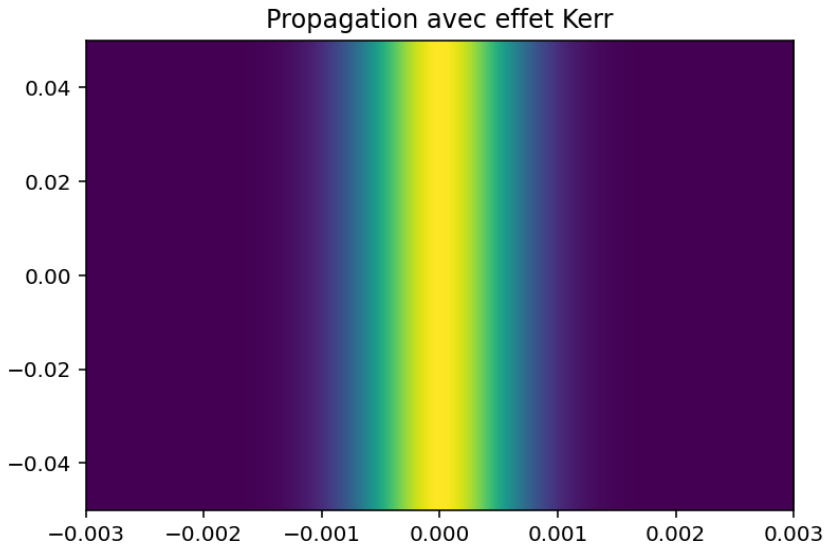

FIGURE 4 – Faisceau avec effet Kerr

L'effet Kerr modifie la propagation en réduisant la divergence, illustrant un début d'auto-focalisation.

6 Discussion

Les résultats obtenus sont en accord avec la théorie des faisceaux gaussiens. La divergence observée est directement liée à la diffraction.

L'introduction de l'effet Kerr montre que les propriétés du milieu influencent fortement la propagation. Bien que le modèle utilisé soit simplifié, il permet de mettre en évidence des phénomènes non linéaires importants.

Dans un contexte réel, ces effets peuvent être amplifiés et conduire à des phénomènes complexes comme la formation de filaments lumineux.

7 Conclusion

Ce projet a permis de modéliser la propagation d'un faisceau gaussien et d'introduire les effets de non-linéarité optique. Les simulations montrent l'importance des paramètres physiques dans la dynamique du faisceau.

Ce travail constitue une base solide pour l'étude de systèmes optiques plus complexes et pour des applications en photonique.

8 Perspectives

- modélisation complète de l'effet Kerr (équation de Schrödinger non linéaire)
- simulation de focalisation réelle
- application à des systèmes laser expérimentaux
- étude de la propagation dans des fibres optiques

9 Annexe : Code Python

```python
import numpy as np
import matplotlib.pyplot as plt

w0 = 1e-3
lambda0 = 633e-9
zR = np.pi * w0**2 / lambda0
I0 = 1

r = np.linspace(-3e-3, 3e-3, 200)
z = np.linspace(-0.05, 0.05, 200)

R, Z = np.meshgrid(r, z)

def w(z):
    return w0 * np.sqrt(1 + (z / zR)**2)

def intensity(r, z):
    return I0 * (w0 / w(z))**2 * np.exp(-2 * r**2 / w(z)**2)

I = intensity(R, Z)

```

```
plt.figure()
plt.plot(r, intensity(r, 0))

plt.figure()
plt.plot(z, w(z))

plt.figure()
plt.imshow(I, extent=[r.min(), r.max(), z.min(), z.max()], aspect='auto')

n2 = 1e-20
I_kerr = I * (1 + n2 * I)

plt.figure()
plt.imshow(I_kerr, extent=[r.min(), r.max(), z.min(), z.max()], aspect='auto')

plt.show()
```